

Hackathon Starter Kit

Build AI apps with AI Foundry Local

Purpose: Enable teams to rapidly prototype secure, on-device AI applications using Local AI Foundry, without cloud dependency.

What is Local AI Foundry?

Local AI Foundry is an on-device AI inference sdk that allows users to run LLMs (magic part!) locally on their own machine through a CLI, SDK, or REST API, without relying on. It is part of the Microsoft Azure AI Foundry ecosystem, but it can also be used independently without a cloud subscription.

It is part of the Azure AI Foundry ecosystem, but can be used fully standalone, making it perfect for POCs and hacking your ideas

AI CoE Principle: Prototype locally → validate value → scale responsibly.

Installation (5 min to first prompt!)

1. open terminal to install the foundry local:

For windows, using the powershell/command :

```
winget install Microsoft.FoundryLocal
```

For mac, using the terminal:

```
brew tap microsoft/foundrylocal
```

```
brew install foundrylocal
```

2. Explore the first local LLMs using the command:

foundry model list

Alias	Device	Task	File Size	License	Model ID
phi-4	GPU	chat-completion	8.83 GB	MIT	phi-4-ovenvino-gpu:1
	GPU	chat-completion	8.37 GB	MIT	phi-4-generic-gpu:1
	CPU	chat-completion	18.16 GB	MIT	phi-4-generic-cpu:1
phi-3.5-mini	GPU	chat-completion	1.95 GB	MIT	Phi-3.5-mini-instruct-ovenvino-gpu:1
	GPU	chat-completion	2.16 GB	MIT	Phi-3.5-mini-instruct-generic-gpu:1
	CPU	chat-completion	2.53 GB	MIT	Phi-3.5-mini-instruct-generic-cpu:1
phi-3-mini-128k	GPU	chat-completion	2.27 GB	MIT	Phi-3-mini-128k-instruct-ovenvino-gpu:1
	GPU	chat-completion	2.13 GB	MIT	Phi-3-mini-128k-instruct-generic-gpu:1
	CPU	chat-completion	2.54 GB	MIT	Phi-3-mini-128k-instruct-generic-cpu:2
phi-3-mini-4k	GPU	chat-completion	2.01 GB	MIT	Phi-3-mini-4k-instruct-ovenvino-gpu:1
	GPU	chat-completion	2.13 GB	MIT	Phi-3-mini-4k-instruct-generic-gpu:1
	CPU	chat-completion	2.53 GB	MIT	Phi-3-mini-4k-instruct-generic-cpu:2
mistral-7b-v0.2	GPU	chat-completion	4.27 GB	apache-2.0	Mistral-7B-Instruct-v0-2-ovenvino-gpu:1
	GPU	chat-completion	4.07 GB	apache-2.0	mistralai-Mistral-7B-Instruct-v0-2-generic-gpu
	CPU	chat-completion	4.07 GB	apache-2.0	mistralai-Mistral-7B-Instruct-v0-2-generic-cpu

3. Example: Qwen 2.5 Coder (1.5B)

foundry model run qwen2.5-coder-1.5b

```
C:\Users\P527961>foundry model run qwen2.5-coder-1.5b
Downloading qwen2.5-coder-1.5b-instruct-ovenvino-gpu:2...
[#####] 100.00 % [Time remaining: about 0s] 3.8 MB/s/s
Unloading existing models. Use --retain true to keep additional models loaded.
🟡 Loading model...
🟢 Model qwen2.5-coder-1.5b-instruct-ovenvino-gpu:2 loaded successfully

Interactive Chat. Enter /? or /help for help.
Press Ctrl+C to cancel generation. Type /exit to leave the chat.
```

```
Interactive mode, please enter your prompt
> What is agentic ai?
🟡 Thinking...
🟢 Agentic AI, also known as AI with consciousness, is a type of artificial intelligence that has the ability to think, reason, and interact with humans in a way that is similar to human intelligence. This means that an agentic AI can perform tasks that require human-like decision-making, such as problem-solving, decision-making, and communication. Agentic AI is a rapidly evolving field, with researchers and developers working on developing more advanced and sophisticated AI systems that can perform a wide range of tasks.
>
```

This gives you a local coding LLM running entirely on your machine

Using Local AI Foundry via Python:

reference: [Foundry Local SDK Reference - Foundry Local | Microsoft Learn](#)

Install dependencies

a. Install dependencies

openai and foudry-local.

Commands in the PowerShell:

pip install openai

pip install foundry-local

b. Using the “ FoundryLocalManager” to create the client:

Minimal Python Example

```
1 import openai
2 from foundry_local import FoundryLocalManager
3 alias = "qwen2.5-0.5b"
4 manager = FoundryLocalManager(alias)
5 client = openai.OpenAI(
6     base_url=manager.endpoint,
7     api_key=manager.api_key # API key is not required for local usage
8 )
9 # Set the model to use and generate a streaming response
10 stream = client.chat.completions.create(
11     model=manager.get_model_info(alias).id,
12     messages=[{"role": "user", "content": "Why is the sky blue?"}],
13     stream=True
14 )
15
16 # Print the streaming response
17 for chunk in stream:
18     if chunk.choices[0].delta.content is not None:
19         print(chunk.choices[0].delta.content, end="", flush=True)
```

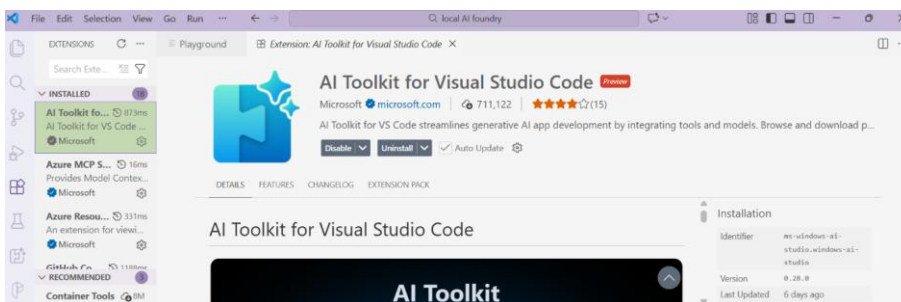
Result:

```
▼ TERMINAL Python + ▢ 🗑️ ...
PS C:\Users\P527961\OneDrive - Nationwide Building Society\Desktop\trials\local AI foundry> & C:/Users/P527961/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/P527961/OneDrive - Nationwide Building Society/Desktop/trials/local AI foundry/local_foundry_test.py"
The sky appears blue because it reflects sunlight. When sunlight enters the Earth's atmosphere, it bounces off the Earth's surface and scatters into the air. The blue color of the sky is due to the scattering of blue light by the atmosphere, which is why we can see the sky as blue.
PS C:\Users\P527961\OneDrive - Nationwide Building Society\Desktop\trials\local AI foundry> ▢
```

- Worth noting, that it uses the same open AI pattern
- No cloud calls needed
- In the future replacements can be dropped in for cloud based frontier LLMs

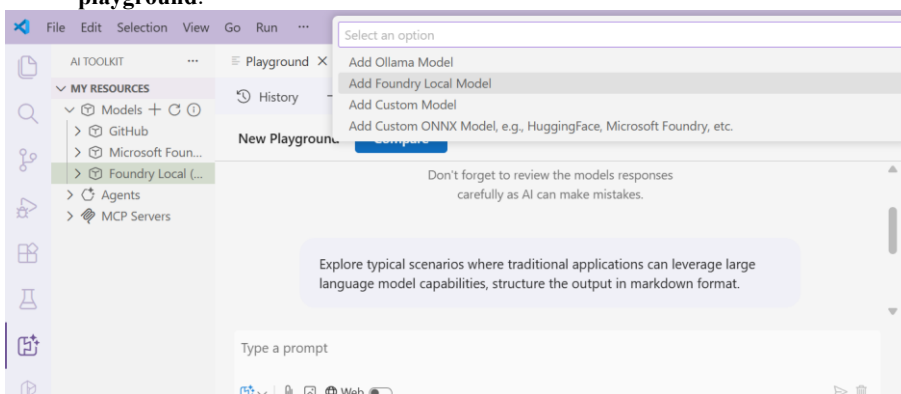
4. Using local AI foundry in VS code extension:

- a. install AI toolkit in the VS code extension:
- b. Search for “AI Toolkit” in VS Code Extensions.

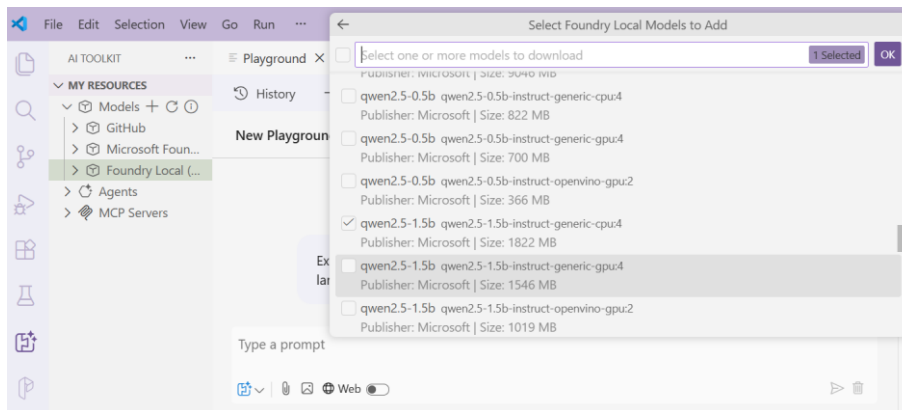


- c. add Foundry Local model in the AI toolkit (Here use qwen2.5-coder-1.5b as an example):

- a. Open AI Toolkit
- b. Add Foundry Local
- c. Select model (e.g. qwen2.5-coder-1.5b) **You can play the local foundry model in the playground:**



d. Prompt, debug, and iterate inside VS Code, fully offline.



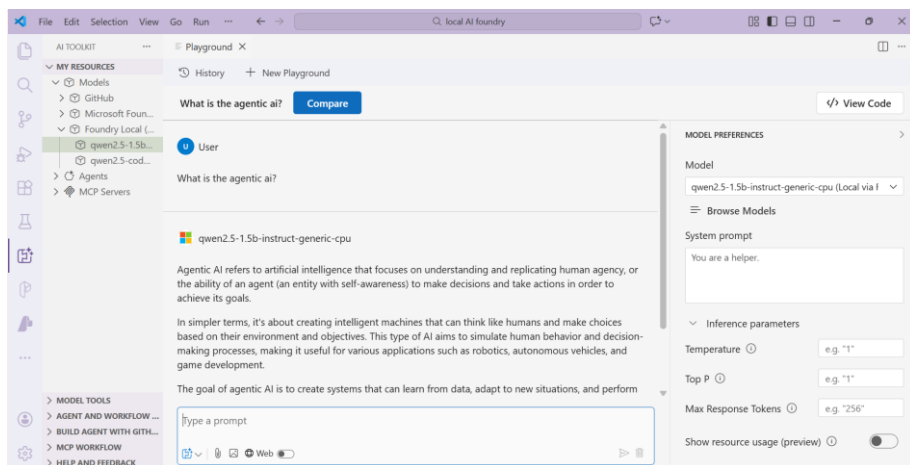
Commented [HC1]: @Meizhen Liu, can you please add how to install vector database details here too.

and then

Commented [HC2]: @Meizhen Liu, can you please add how to install vector database details here too.

and then

Commented [ML3R2]: Yes! All done. Please feel free to give any constructive feedback.



6. Combine Local AI Foundry with grouping with your data

Install vector store

install chromadb:

Command in the PowerShell:

pip install chromadb

Simple Local RAG Flow

Documents → Chunking → Embeddings → ChromaDB



User Query → Retrieve Context → Prompt → Local LLM

[2] Select an embedding model to vectorize the text chunks, then use ChromaDB to create a collection for storing the resulting vectors.

This example uses the default embedding function, which relies on the Sentence Transformers all-MiniLM-L6-v2 model.

Feel free to substitute it with any embedding model you prefer.

```
1 import chromadb
2 from chromadb.utils.embedding_functions import DefaultEmbeddingFunction
3
4 client = chromadb.Client()
5
6 default_ef = DefaultEmbeddingFunction()
7 collection = client.create_collection(
8     name="docs",
9     embedding_function=default_ef
10 )
11
12 collection.add(
13     ids=["1", "2", "3", "4", "5"],
14     documents=[
15         "Deep learning uses neural networks with many layers to model complex relationships.",
16         "Hello world example for testing embeddings.",
17         "ONNX Runtime enables high performance for machine learning models.",
18         "Machine learning is a subset of artificial intelligence focused on building systems that learn from data.",
19         "Tokenization is the process of converting text into smaller units called tokens.",
20     ],
21 )
22
23
```

[3] Use collection.query to retrieve the relevant chunks, incorporate those chunks into the prompt as contextual information, and then pass the completed prompt to the local LLM.

```
1 system_prompt="""
2 You are an AI assistant. Use ONLY the following context to answer the question.
3 Context:
4 {context}
5 """
6 user_prompt="""
7 Question: {question}
8 """
```

[4] Result:

```
Enter your question: What is deep learning?
Retrieved context: [['Deep learning uses neural networks with many layers to model complex relationships.',
'Machine learning is a subset of artificial intelligence focused on building systems that learn from data.']]
]
Deep learning is a subset of machine learning that focuses on training models to learn complex patterns and
relationships in data. It involves using neural networks with multiple layers to learn and make predictions
on large datasets. Deep learning algorithms are designed to be able to recognize and learn from patterns in
data that would be difficult or impossible for traditional machine learning algorithms to process.
```

```
1  import openai
2  from foundry_local import FoundryLocalManager
3  from prompt import system_prompt, User_prompt
4  from local_rag import collection
5
6  # Set up the Foundry Local Manager and OpenAI client
7  alias = "qwen2.5-0.5b"
8  manager = FoundryLocalManager(alias)
9  client = openai.OpenAI(
10     base_url=manager.endpoint,
11     api_key=manager.api_key # API key is not required for local usage
12 )
13
14 # get user input and retrieve context from ChromaDB
15 user_input = input("Enter your question: ")
16 context = collection.query(
17     query_texts=[user_input],
18     n_results=2,
19 )
20 print("Retrieved context:", context["documents"], flush=True)
21
22
23 # Set the model to use and generate a streaming response
24 stream = client.chat.completions.create(
25     model=manager.get_model_info(alias).id,
26     messages=[{"role": "system", "content": system_prompt}, {"role": "user", "content": User_prompt.format(question=user_input, context=context["documents"])},
27     stream=True
28 )
29
30 # Print the streaming response
31 for chunk in stream:
32     if chunk.choices[0].delta.content is not None:
33         print(chunk.choices[0].delta.content, end="", flush=True)
34
35
```

Recommended Architecture Pattern for hack idea

Local AI Foundry → Local RAG (Chroma DB) → Validate usecase

This starter kit is designed to remove friction and maximize learning velocity.

Focus on use-case clarity, not infra complexity. Happy hacking!